



**TECNOLÓGICO
NACIONAL DE MÉXICO**



Administración y Organización de Datos

No. Alumno:

Christian Daniel Martínez Márquez

No. Control:

242310396

Docente:

Jesús Salas Marín

1. El Enfoque del Proyecto: Romper la Barrera del Millón de Registros

En este cuarto corte, dejamos atrás los sistemas web tradicionales para adentrarnos por completo en el mundo del Big Data y la optimización de algoritmos. El verdadero reto aquí no fue solo hacer que un programa funcione, sino lograr que no colapse la memoria RAM ni el procesador de la computadora al enfrentarse a volúmenes masivos de datos hospitalarios (simulando desde 1 millón hasta 20 millones de registros de pacientes). A mis 19 años, este proyecto cambió por completo mi perspectiva sobre el desarrollo de software, obligándome a diseñar una arquitectura modular y eficiente que prioriza el rendimiento, la velocidad de entrada/salida (I/O) y la paridad de la información bajo estándares críticos de la industria de la salud.

2. El Cerebro Estructural y la Coordinación Dinámica del Sistema (menu.py y generador_datos.py)

La columna vertebral de la aplicación está dividida en dos pilares de control. Por un lado, menu.py actúa como la interfaz de consola centralizada que coordina de forma limpia todas las opciones del usuario, desde la carga inicial hasta las búsquedas y visualizaciones de reportes, manteniendo variables compartidas sin sobrecargar el entorno global. Por el otro lado, para poder probar el estrés del sistema, desarrollé generador_datos.py. Este script es una máquina automatizada que crea perfiles médicos ficticios pero realistas (nombres, edades, temperaturas, presiones arteriales, especialidades y medicamentos). Lo interesante de este generador es que incluye un sistema de cronómetro interno (time.time()) que calcula los segundos exactos que toma escribir estos datos masivos y mide el peso de los archivos generados en disco, dándonos una base sólida para realizar pruebas de rendimiento reales.

3. Ingeniería de Datos y Estrategias contra el Agotamiento de Memoria (datos.py y pacientes.py)

La manipulación de los archivos es donde implementé las técnicas de optimización más avanzadas del proyecto a través de datos.py y pacientes.py. Si intentáramos cargar un archivo de un millón de líneas de golpe, la memoria RAM se saturaría de inmediato. Para resolverlo, en datos.py utilicé la técnica de lectura por bloques utilizando la propiedad chunksize=100_000 de Pandas para el archivo datos_1000000.csv, procesando porciones de información de manera secuencial para acumular sumas, promedios, máximos y mínimos. En el caso de datos_1000000.json, diseñé un algoritmo nativo que lee y parsea línea por línea mediante json.loads(). Además, en pacientes.py aseguré la paridad de datos al permitir agregar nuevos pacientes en tiempo real tanto en el archivo CSV como en

el JSON, aplicando controles binarios preventivos que revisan si el archivo termina con un salto de línea para evitar la corrupción de las columnas. Para las búsquedas, el sistema también escanea por bloques, deteniéndose al encontrar las primeras coincidencias para optimizar el tiempo de respuesta.

4. Analítica Avanzada, Reportes Clínicos y Dashboards Visuales (estadisticas.py y graficas.py)

Una vez que los datos son procesados de forma eficiente, el sistema los transforma en conocimiento útil para la toma de decisiones médicas y administrativas a través de `estadisticas.py` y `graficas.py`. El script `estadisticas.py` se encarga de dar formato y desplegar en la consola indicadores clínicos críticos como la edad promedio de los pacientes, rangos de temperatura para control de fiebre, un Top 5 de los medicamentos más recetados y el volumen de consultas por especialidad. Para llevar esto a un nivel ejecutivo, `graficas.py` aprovecha la potencia de Matplotlib y los métodos directos de visualización de Pandas para renderizar de forma secuencial un histograma de edades, gráficas de barras para las consultas, un gráfico de dispersión que correlaciona la temperatura con la edad, y un gráfico de pastel con los porcentajes de medicamentos, además de generar ventanas comparativas para analizar visualmente el tiempo y peso que toma procesar CSV contra JSON.

5. Conclusión y Visión de Escalabilidad en Entornos de Producción

Desarrollar este sistema de optimización masiva me demostró que el verdadero valor de un ingeniero no es solo escribir código, sino saber administrar los recursos físicos del hardware de manera inteligente. Logré conectar la modularidad de Python con análisis estadísticos avanzados y técnicas de manipulación de archivos que se usan en entornos reales de Big Data. La arquitectura que diseñé es totalmente escalable: si la clínica o el hospital crecieran al grado de manejar cientos de millones de registros médicos, la lógica analítica de bloques que implementé serviría como el puente perfecto para migrar la persistencia hacia un clúster de bases de datos distribuidas o herramientas de la nube como Apache Spark o Google BigQuery, manteniendo el menú, las validaciones y los módulos de graficación intactos.